

Accessible Dining Finder for Dietary Needs: Revised Project Spec

General Description of the Project (2 pts)

- Traditional restaurant discovery tools focus on general ratings and reviews but do not reliably surface information about menu details.
- This application aims to fill that gap by extracting and analyzing relevant information to generate a “dietary safety score” for each restaurant.
- The application will allow users to input a location (such as a ZIP code or city or county) to discover nearby dining options.
- Restaurants will be displayed on an interactive map with markers indicating their location.
- Selecting a restaurant will reveal additional details, including ~~extracted review snippets mentioning dietary accommodations~~ (e.g., “gluten-free options,” “vegan options”) and a computed safety score based on ~~keyword~~ dietary tag analysis.
- Users will be able to filter results based on criteria such as dietary needs (gluten-free, vegan), ~~cuisine type~~, and distance from their location.
- External:
 - ~~Nominatum API for retrieving restaurant location data~~
 - Overpass API for retrieving restaurant ~~location and~~ metadata
 - Streamlit for building the user interface
 - Folium for rendering interactive maps
 - ~~Render URL for making the app mobile friendly~~
 - ~~Basic keyword matching for review analysis~~
- GUI: Web app using Streamlit, but initial development may simulate interactions via a Jupyter Notebook or CLI. ~~Also utilizing Render URL to view on mobile.~~
- Possible Enhancements:
 - ~~More advanced NLP (sentiment detection, more thorough menu analysis)~~
 - ~~Users contribute their own notes or experiences, stored locally to enhance the dataset (SQLite or CSV for storing user-generated data)~~
 - Clarification of dietary safety score
 - Mobile UI improvements (color, content, app name, etc)
 - More detailed/clear clickable popups on folium map

Task Vignettes (User Activity “Flow”) (4 pts)

- Task 1: Search for Restaurants by Location
 - User configurable fields:
 - Location
 - Dietary needs
 - Radius
- Task 1 User Activity:
 - User opens app and is prompted to enter a location (Input: location string)

- User enters mile radius from selected location
- User enters dietary needs (gluten free, dairy free, and/or vegan)
- User clicks "Search," and app displays an interactive map populated with restaurant markers (Query Overpass API for nearby restaurants, output: map markers, store results in a dictionary)
- User can click into map pins to view restaurant names, approximate distance, and a safety score
- Task 2: Apply Filters
 - After viewing initial results, user can apply filters
 - User configurable fields
 - Dietary Needs
 - Distance
 - Cuisine
- Task 2 User Activity:
 - User applies dietary filter "Gluten-free" (Input: Dietary keyword filter)
 - User adjusts distance within a certain mile radius (Numeric input, validation: distance must be positive)
 - User specifies cuisine type (Dropdown or text filter)
 - User clicks "Apply Filters," and the map updates. Restaurants that do not meet the criteria are removed, and the list view updates accordingly. Filtering occurs on pre-fetched dataset (no new API call)
- Task 3: View Restaurant Details and Safety Score
 - Restaurant details display when the user clicks on a restaurant pin on the map. This helps the user quickly assess whether the restaurant is safe for their needs.
- Task 3 User Activity:
 - User clicks on a restaurant pin
 - User views restaurant name, location, computed "safety score" (safety score based on frequency of keyword metadata dietary tag matching ("gluten free"), output stored as part of restaurant dict)

Technical "Flow" (3 pts)

Data Structures: Restaurant dataset is a list of dictionaries OR pandas DataFrame with fields including name, latitude/longitude, cuisine (if available), raw tags/description, safety_score, and extracted_keywords. The user input config (dict) contains the location, radius, dietary_filters, and cuisine_filter.

Core Functions:

- def get_location(user_input):
 - Convert location string to lat/long (or use default)
 - Uses Nominatum API
 - return lat, lon, and a bounding box
- def fetch_restaurants(lat, lon, radius, bounding box):

- Call Overpass API
- Parse JSON response
- return restaurant_list dictionary
- def compute_safety_score(restaurant):
 - Keyword matching on tags/menus
 - Assign score based on presence of keywords
 - return score
- def filter_restaurants(data_restaurants, filters):
 - Apply dietary, distance, cuisine filters
 - return filtered_data
- def generate_map(data):
 - Create Folium map
 - Add markers with popups
 - return folium_map

Program Flow:

- User launches Streamlit app
- App collects location, dietary needs, radius input
- get_location() resolves coordinates
- fetch_restaurants() retrieves data from Overpass API
- For each restaurant, compute_safety_score() is applied
- Data stored in DataFrame dictionary
- User applies filters with filter_restaurants()
- generate_map() renders results
- User interacts with markers and then details are displayed
- API fetch, then parsing, then keyword metadata analysis, then scoring, then filtering
- Output: Map (Folium), detail panel popup (details and score)

Final (Self) Assessment (1 pt)

- I refined how the “safety score” is calculated and scoped it down to keyword-based analysis rather than full NLP. I also clarified that filtering will initially occur on a pre-fetched dataset rather than querying the API multiple times.
- I feel moderately confident implementing API integration, filtering logic, and a Streamlit interface.
- I decided to go with Streamlit instead of Flask after watching the lecture video comparing the two options, since Streamlit seemed more understandable for me.
- The biggest risk is the depth of data available from the Overpass API. It may not include rich review text, which could limit how meaningful the safety score can be (items labeled “gluten free” on menus may be exposed to cross contamination in the kitchen which may affect safety for folks with Celiac disease)
- May need help optimizing speed/performance when displaying a map with multiple pins